FeignClient

Friday, 6 June 2025 5:29 PM

FeignClient

- Feign is a Declarative HTTP client developed by Netflix.
- Declarative means "we tell What to do, not How to do".

In SpringBoot , Feign capability is available via Spring Cloud OpenFe

- Spring Cloud provides a set of tool and libraries, which helps to build distrib
- As it provides seamless integration with :
  - Service Discovery
  - Client side Load Balancing
  - Circuit Breaker and Resilience
  - Api Gateway
  - Distributed Tracing
  - Centralized Configuration etc....

Pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

In future, we might use more Spring Cloud libraries (for Load balancer, for Se
Spring Cloud libraries should have compatible version, therefore we use belo
we don't have to mange it manually.

That's why we are not specifying the version with above "spring-cloud-st
it will be taken care by our dependency management.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.1</version> <!-- Use latest comp
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

OrderService

(running on localhost:8081)

OrderService needs to invoke ProductService

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}")
public interface ProductClient {

    @GetMapping("/products/{id}")
```

```
    String getProductById(@PathVariable("id") String id);

}
```

We are not writing any logic, just told what to call.

application.properties

```
server.port=8081

#Base URL for Product Service
feign.client.product-service.url=http://localhost:8082
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    ProductClient productClient;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        String responseFromProductAPI = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + responseFromP

        return ResponseEntity.ok( body: "order call successful");
    }
}
```

```
@SpringBootApplication
@EnableFeignClients
public class OrderserviceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class
    }
}
```

It enable Feign support and tells S
for interfaces annotated with @F

Start the application and invoke the Order Endpoint

GET localhost:8081/orders/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

	Key
	Key

Body Cookies Headers (5) Test Results

Raw Preview Visualize

```
1  order call successful
```

```
2025-06-06T21:11:17.438+05:30  INFO 18774 --- [          main] c.c.o.OrderserviceApplication
2025-06-06T21:11:17.439+05:30  INFO 18774 --- [          main] c.c.o.OrderserviceApplication
2025-06-06T21:11:17.705+05:30  INFO 18774 --- [          main] o.s.cloud.context.scope.GenericSc
2025-06-06T21:11:17.810+05:30  INFO 18774 --- [          main] o.s.b.w.embedded.tomcat.TomcatWeb
2025-06-06T21:11:17.814+05:30  INFO 18774 --- [          main] o.apache.catalina.core.StandardSe
2025-06-06T21:11:17.814+05:30  INFO 18774 --- [          main] o.apache.catalina.core.StandardEn
```

```
2025-06-06T21:11:17.836+05:30  INFO 18774 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/
2025-06-06T21:11:17.837+05:30  INFO 18774 --- [           main] w.s.c.ServletWebServerApplication
2025-06-06T21:11:17.999+05:30  INFO 18774 --- [           main] o.s.b.w.embedded.tomcat.TomcatWeb
2025-06-06T21:11:18.005+05:30  INFO 18774 --- [           main] c.c.o.OrderserviceApplication
2025-06-06T21:11:28.409+05:30  INFO 18774 --- [nio-8081-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/
2025-06-06T21:11:28.409+05:30  INFO 18774 --- [nio-8081-exec-2] o.s.web.servlet.DispatcherServlet
2025-06-06T21:11:28.410+05:30  INFO 18774 --- [nio-8081-exec-2] o.s.web.servlet.DispatcherServlet
Response from Product api call is: fetch the product details with id:1
```

So, first important thing to understand is, how this Declarative H

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}")
public interface ProductClient {

    @GetMapping("/products/{id}")
    String getProductById(@PathVariable("id") String id);

}
```

We have not provide any implementation, but how come we are able to Autowired it, without any exception?

```
@RestContro
@RequestMap
public clas
    @Autowi
    Product
    @GetMap
    public
        Str
        Sys
        ret
    }
}
```

Application Startup → *Scan for @FeignClient interface* → ***For each i***

ReflectiveFe → 1. For each method in an interface → *Create an Object of MethodHandler and holds it in Map<Method, MethodHandler>*

MethodHandler *object holds complete info that is parsed from a method and its annotations, which is needed to execute the HTTP request.*

Below are some of the fields which ***MethodHandler*** *object holds:*

- Target URL: info regarding Base URL + relative path
  - @FeignClient(name="product-service", *url="http://localhost:8082"*)
  - @GetMapping(*"/products/{id}"*)

Creates an Object of I
populate the variable:
which we get in #1.

InvocationHandler, ac
- *Proxy Implementat*
  and

- HTTP Method: GET, PUT, POST etc.
  - Derived from *@GetMapping*, *@PostMapping* annotations etc.

- Header Info
  - Derived from *@RequestHeader* or *interceptors* etc.

- HTTP Client: actual client to make a HTTP call.
  - We can configure to either choose:
    - *HttpURLConnection* (default)
    - *OkHttp*
    - *ApacheHttpClient*

- Encoder: Converts request body from Java object to HTTP format like JSON
  - Configurable, we can provide our own implementation

- Decoder: Converts response to Java object
  - Configurable, we can provide our own implementation

- ErrorDecoder: Tells what to do, when exception is thrown
  - Configurable, we can provide our own implementation

- Logger: logs the request and response info.
  - Configurable, we can provide our own implementation

- Retryer: handle retries when failure happens.
  - Configurable, we can provide our own implementation

- *The MethodHandle*
  invoke the HTTP ca

```
@Override
public Object invoke
{
    // Step 1: Look u
    MethodHandler

    // Step 2: Use M
    return methodH
}
```

***MethodHandler** also has method invoke(), which knows how to create HTTP request object from above info a*

```
@Override
public Object invoke(Object[] argv) throws Throwable {
    RequestTemplate template = buildTemplateFromArgs.create(argv);
    Options options = findOptions(argv);
    Retryer retryer = this.retryer.clone();
    while (true) {
        try {
            return executeAndDecode(template, options);
```

```
        } catch (RetryableException e) {
            try {
                retryer.continueOrPropagate(e);
            } catch (RetryableException th) {
                Throwable cause = th.getCause();
                if (propagationPolicy == UNWRAP && cause != null) {
                    throw cause;
                } else {
                    throw th;
                }
            }
            if (logLevel != Logger.Level.NONE) {
                logger.logRetry(metadata.configKey(), logLevel);
            }
            continue;
        }
    }
}
```

An example below, with various annotation usage demo like @GetMa @RequetParam, @RequestHeader, @RequestBody etc..

Order Application

```
@FeignClient(name = "product-service", url = "${feign.client.product-servic
public interface ProductClient {

    @GetMapping("/products/{id}")
    String getProductById(@PathVariable("id") String id);

    @PutMapping(value = "/products/update/{id}", consumes = "application/js
    Product updateProduct(
            @PathVariable("id") String id,
            @RequestParam("sendMail") boolean sendMail,
            @RequestHeader("X-ConceptCoding-ID") String uniqueID,
            @RequestBody Product updatedProductDetails
```

Ordering is r
take care to

```
    );
}
```

annotations.

Encoder and Decoder in FeignClient

- **Encoder**: Converts a Java object into a request body (say JSON).
- **Decoder**: Converts the HTTP response body (say JSON) into a Ja

```
@Override
public Object invoke(Object[] argv) throws Throwable {
    RequestTemplate template = buildTemplateFromArgs.create(argv);
    Options options = findOptions(argv);
    Retryer retryer = this.retryer.clone();
    while (true) {
        try {
            return executeAndDecode(template, options);
        } catch (RetryableException e) {
            try {
                retryer.continueOrPropagate(e);
            } catch (RetryableException th) {
                Throwable cause = th.getCause();
                if (propagationPolicy == UNWRAP && cause != null) {
                    throw cause;
                } else {
```

```
                    throw th;
                }
            }
            if (logLevel != Logger.Level.NONE) {
                logger.logRetry(metadata.configKey(), logLevel);
            }
            continue;
        }
    }
}
```

If we want our custom Encoder and Decoder implementation:

All custom configuration defined in Pro
this ProductClient only.

We can have many ClientConfig like Sa
CustomerClientConfig etc. with their ov
implementation. Not impacting each o

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}",
        configuration = ProductClientConfig.class)
public interface ProductClient {

    @PutMapping(value = "/products/update/{id}", consumes = "application/j
    Product updateProduct(
            @PathVariable("id") String id,
            @RequestParam("sendMail") boolean sendMail,
            @RequestHeader("X-ConceptCoding-ID") String uniqueID,
            @RequestBody Product updatedProductDetails
    );
}
```

```
@Configuration
public class ProductClientConfig {

    @Bean
    public Encoder myCustomEncoder() {
        return new MyCustomProductClientEncoder();
    }

    @Bean
```

```
    @Bean
    public Decoder myCustomDecoder() {
        return new MyCustomProductClientDecoder();
    }
}
```

ErrorDecoder in FeignClient

- It is used to handle non 2xx status codes like 4xx and 5xx.

```
@Override
public Object invoke(Object[] argv) throws Throwable {
    RequestTemplate template = buildTemplateFromArgs.create(arg
    Options options = findOptions(argv);
    Retryer retryer = this.retryer.clone();
    while (true) {
        try {
            return executeAndDecode(template, options);
        } catch (RetryableException e) {
            try {
                retryer.continueOrPropagate(e);
            } catch (RetryableException th) {
                Throwable cause = th.getCause();
```

```
                if (propagationPolicy == UNWRAP && cause != nu
                    throw cause;
                } else {
                    throw th;
                }
            }
            if (logLevel != Logger.Level.NONE) {
                logger.logRetry(metadata.configKey(), logLevel)
            }
            continue;
        }
    }
}
```

If we want our custom Erro

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}",
        configuration = ProductClientConfig.class)
public interface ProductClient {

    @PutMapping(value = "/products/update/{id}", consumes = "application/json")
    Product updateProduct(
            @PathVariable("id") String id,
            @RequestParam("sendMail") boolean sendMail,
            @RequestHeader("X-ConceptCoding-ID") String uniqueID,
            @RequestBody Product updatedProductDetails
    );
}
```

```
@Configuration
public class ProductClientConfig {

    @Bean
```

```
    public ErrorDecoder myCustomErrorDecoder() {
        return new MyCustomProductClientErrorDecoder();
    }
}
```

```
2025-06-07T20:26:24.770+05:30  INFO 24295 --- [nio-8081-exec-1] o.s.web.servlet
2025-06-07T20:26:24.812+05:30 ERROR 24295 --- [nio-8081-exec-1] o.a.c.c.C.[.[.

com.conceptandcoding.orderservice.BadRequestException Create breakpoint : Client Er
    at com.conceptandcoding.orderservice.MyCustomProductClientErrorDecoder.deco
    at feign.InvocationContext.decodeError(InvocationContext.java:126) ~[feign-
    at feign.InvocationContext.proceed(InvocationContext.java:72) ~[feign-core-
    at feign.ResponseHandler.handleResponse(ResponseHandler.java:63) ~[feign-co
```

Retryer in FeignClient

```
@Override
public Object invoke(Object[] argv) throws Throwable {
    RequestTemplate template = buildTemplateFromArgs.create(a
    Options options = findOptions(argv);
    Retryer retryer = this.retryer.clone();
    while (true) {
        try {
            return executeAndDecode(template, options);
        } catch (RetryableException e) {
            try {
```

```
                retryer.continueOrPropagate(e);
            } catch (RetryableException th) {
                Throwable cause = th.getCause();
                if (propagationPolicy == UNWRAP && cause != n
                    throw cause;
                } else {
                    throw th;
                }
            }
            if (logLevel != Logger.Level.NONE) {
                logger.logRetry(metadata.configKey(), logLeve
            }
            continue;
        }
    }
}
```

- Retry only happens when there is either:
  - Connection time out
  - Network related exception like (IOException)
- After all retry finished, then ErrorDecoder is invoked.
- For 4xx and 5xx, retry do not happens, its handled by ErrorDe

```
public interface Retryer extends Cloneable {

    if retry is permitted, return (possibly after sleeping). Otherwise, propagate the exceptio
    void continueOrPropagate(RetryableException e);

    Retryer clone();

    class Default implements Retryer {

        private final int maxAttempts;
        private final long period;
        private final long maxPeriod;
        int attempt;
        long sleptForMillis;

        public Default() {        this( period: 100, SECONDS.toMillis( duration:
```

- Defau
  - 5 t
  - Ini
  - Wa
  - Bu

- Try
- Try
- Try
- Try
- Try

Aft

If, we don't want to retry at all, we can use Retryer.NEVER

```
@Configuration
public class ProductClientConfig {

    @Bean
    public Retryer myCustomRetryer() {
        return Retryer.NEVER_RETRY;
    }
}
```

If, we want custom implementatic

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service.url}",
        configuration = ProductClientConfig.class)
public interface ProductClient {

    @PutMapping(value = "/products/update/{id}", consumes = "application/json")
    Product updateProduct(
            @PathVariable("id") String id,
            @RequestParam("sendMail") boolean sendMail,
            @RequestHeader("X-ConceptCoding-ID") String uniqueID,
            @RequestBody Product updatedProductDetails
    );
}
```

```java
@Configuration
public class ProductClientConfig {

    @Bean
    public Retryer myCustomRetryer() {
        return new MyCustomRetryer();
    }
}
```

Last but not the least:

During start, we discussed that, this name is just a arbitrary v

giving the name to our FeignClient.

```
@FeignClient(name = "product-service",
        url = "${feign.client.product-service
public interface ProductClient {

    @GetMapping("/products/{id}")
    String getProductById(@PathVariable("id")

}
```

But where its exactly used?

Yes, this name comes handy, when we have to provide any configur
application.properties

If we want to set request and connection timeout only for produ

application.properties

```
#request and connection timeout applicable to only Product
feign.client.config.product-service.connectTimeout=3000
feign.client.config.product-service.readTimeout=5000
```

If we want to set request and connection timeout for all FeignClient

application.properties

```
#request and connection timeout applicable for all FeignC
feign.client.config.default.connectTimeout=3000
feign.client.config.default.readTimeout=5000
```

[redacted]